

adiabatic

May 5, 2026

1 Ising Adiabatic State Preparation

1.1 *Review:* Postulate 6

The time evolution of the state vector $|\psi(t)\rangle$ is governed by the Schrödinger equation:

$$i\hbar \frac{d}{dt} |\psi(t)\rangle = \hat{H}(t) |\psi(t)\rangle,$$

1.1.1 Time independent case

Imagine the case where $\hat{H}(t) \neq f(t)$. This is just a statement that the Hamiltonian operator doesn't change in time. If that is the case, then we can directly integrate this differential equation, to get the following solution:

$$\begin{aligned} |\psi(t)\rangle &= e^{-\frac{i}{\hbar} \hat{H} \Delta t} |\psi(t_0)\rangle, \\ &= U(t, t_0) |\psi(t_0)\rangle \end{aligned}$$

where $\Delta t = t - t_0$. Differentiate this wavefunction to convince yourself that this is a solution to the time dependent Schrödinger equation.

Because of this simple form, we can do fancy things, like first go to t_1 , then go to t_2 , e.g.,

$$|\psi(t_2)\rangle = U(t_2, t_1) U(t_1, t_0) |\psi(t_0)\rangle$$

1.1.2 Time dependent case

The more general (and complicated) case where $\hat{H}$ is time-dependent cannot be written down in such a simple form. However, we can develop an arbitrarily accurate approximation to the solution by using the result above. Assume we want to solve for $|\psi(t)\rangle$, we can break up the evolution into *infinitely* many steps:

$$|\psi(t)\rangle = \lim_{N \rightarrow \infty} U(t, t_{N-1}) \dots U(t_2, t_1) U(t_1, t_0) |\psi(t_0)\rangle$$

By dividing the evolution into small enough time steps, we can use the time independent solution! The core idea, is that within a small enough time window, $t_i \rightarrow t_{i+1}$, the Hamiltonian is changing slowly enough such that it appears constant. This means that whatever the Hamiltonian looks like at a give time, ($\hat{H}(t_i)$), we can exponentiate it, to obtain a unitary $U(t_{i+1}, t_i)$.

In practice, of course, we can never write down a real ∞ of terms. And so we simply stop at some point and accept that there is some approximation being made.

$$|\psi(t)\rangle \approx U(t, t_{N-1}) \dots U(t_2, t_1) U(t_1, t_0) |\psi(t_0)\rangle$$

We often refer to this particular approximate form of the time evolution as being **trotterized**

2 Adiabatic Principle

We can use the above result (the “trotterized” form of the TDSE solution) to solve a generally difficult problem, **finding the ground state of a Hamiltonian**. In our Ising case, this amounts to finding the lowest energy spin configuration out of an exponentially large number of candidates. However, if we were to be able to find the ground state, we would simultaneously be solving a wide range of extremely important problems including the travelling salesman problem or finding the max-cut of a graph or network.

In this notebook, we will use the **adiabatic principle** to define an interesting (yet simple) quantum algorithm for obtaining arbitrarily accurate approximations to Hamiltonian ground states.

We will define the adiabatic principle as follows: >If we start out in the ground state of a Hamiltonian $\hat{H}_A$, and the *slowly* move to Hamiltonian $\hat{H}_B$, then we will end up in the ground state of $\hat{H}_B$ (assuming we move slow enough and there is a gap).

To use the adiabatic principle to obtain the ground state for a target Hamiltonian, $\hat{H}_B$, (which could be the Ising example we have been working with or some other problem) we will define the following time dependent Hamiltonian:

$$\hat{H}(t) = (1 - f(t))\hat{H}_A + f(t)\hat{H}_B,$$

where $\hat{H}_A$ is our starting Hamiltonian (this should be an easy Hamiltonian for which we know the ground state). Notice that while both $\hat{H}_A$ and $\hat{H}_B$ are time-*independent*, the coefficient $f(t)$ changes in time, so the total Hamiltonian is time-*dependent*.

This means that we can define any function $f(t)$ that starts at 0 and ends up equal to 1 at some final time, t_f . - $f(t=0) = 0$ - $f(t=T) = 1$

The simplest time-dependent function that we might consider is just a linear interpolation between the two Hamiltonians:

- $f(t) = t/T$ where T is the total time of evolution. This means that a larger T results in a *slower* evolution, and satisfies the condition that $\hat{H}(0) = \hat{H}_A$ and $\hat{H}(T) = \hat{H}_B$.

2.1 Trotterization

In addition to requiring that our final time T is large enough to ensure that our process is slow, we also need to make sure that we are solving the resulting time dependent schrödinger equation accurately enough. Since we are using the trotter approximation (meaning treating the total evolution as a sequence of short time-independent evolutions), we will also need to choose a number of time steps N to be large enough for the adiabatic evolution to be accurate.

Each time step is evolved by a time step given by: - $\Delta t = T/N$

We will want both T and N to be large enough for accurate results.

$$\begin{aligned} |\psi(T)\rangle &\approx U((N+1)\Delta t, N\Delta t) \cdots U(2\Delta t, \Delta t) U(\Delta t, 0) |\psi(0)\rangle \\ |\psi(T)\rangle &\approx \prod_{k=0}^{N-1} U((k+1)\Delta t, k\Delta t) |\psi(0)\rangle = \prod_{k=0}^{N-1} U_k |\psi(0)\rangle \end{aligned}$$

$$U_k = e^{-i\Delta t \hat{H}(k\Delta t)}$$

Since we have an explicit form of our time dependent hamiltonian $\hat{H}(t)$, we can write this out:

$$\hat{H}(k\Delta t) = (1 - \frac{k\Delta t}{T})\hat{H}_A + \frac{k\Delta t}{T}\hat{H}_B$$

Convince yourself that when $k = 0$, we have the initial hamiltonian, H_A , and when $k = N$, we have the final Hamiltonian, H_B

2.2 One more trotterization

$$\begin{aligned} U_k &= e^{-i\Delta t \hat{H}(k\Delta t)} \\ &= e^{-i\Delta t ((1 - \frac{k\Delta t}{T})\hat{H}_A + \frac{k\Delta t}{T}\hat{H}_B)} \\ &\approx e^{-i\Delta t (1 - \frac{k\Delta t}{T})\hat{H}_A} e^{-i\Delta t \frac{k\Delta t}{T}\hat{H}_B} \end{aligned}$$

Using $T = 2$ and $N = 4$, (such that $\Delta t = \frac{1}{2}$) from the case above we can define the solution as:

$$\begin{aligned} |\psi(2)\rangle &= e^{-i\Delta t (\frac{1}{4}\hat{H}_A + \frac{3}{4}\hat{H}_B)} e^{-i\Delta t (\frac{1}{2}\hat{H}_A + \frac{1}{2}\hat{H}_B)} e^{-i\Delta t (\frac{3}{4}\hat{H}_A + \frac{1}{4}\hat{H}_B)} e^{-i\Delta t \hat{H}_A} |\psi(0)\rangle \\ &= U(2, \frac{3}{2}) U(\frac{3}{2}, 1) U(1, \frac{1}{2}) U(\frac{1}{2}, 0) \\ &\approx e^{-i\Delta t \frac{1}{4}\hat{H}_A} e^{-i\Delta t \frac{3}{4}\hat{H}_B} e^{-i\Delta t \frac{1}{2}\hat{H}_A} e^{-i\Delta t \frac{1}{2}\hat{H}_B} e^{-i\Delta t \frac{3}{4}\hat{H}_A} e^{-i\Delta t \frac{1}{4}\hat{H}_B} e^{-i\Delta t \hat{H}_A} |\psi(0)\rangle \end{aligned}$$

We can use a quantum computer to carry out these operations. But first we will need to know how to represent these quantities on a quantum computer.

3 Quantum Representation of Ising

In the previous examples, we have used ‘classical’ bitstrings, by which I mean all of our states are always written as a single product of zeros and ones, i.e., $|010010\rangle$. So far, we have used the Ising Hamiltonian,

$$\hat{H} = \sum_{(i,j) \in E} J_{ij} s_i s_j + \sum_i \mu_i s_i,$$

to compute the energy of these bitstrings where the spin function takes in a bit and returns either 1 or -1. For example, if $|\psi\rangle = |0101\rangle$, then $s_2 |0101\rangle = 0$.

We can make this a bit more mathematically useful, and define our quantities in terms of linear algebra, making our spin function and operator, and the state a vector. Each bit is two-dimensional (it can be either zero or one), so we can express it as a 2-dimensional vector:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Similarly, our “spin-functions”, s_i , can also be expressed in terms of a simple, linear algebra quantity, $\hat{z}$

$$\hat{z} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

Consider the following operator, $\hat{x}$:

$$\hat{x} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

What happens when we apply $\hat{x}$ to our states?

$$\begin{aligned} \hat{x} |0\rangle &= |1\rangle \\ \hat{x} |1\rangle &= |0\rangle \end{aligned}$$

Here, $\hat{z}$ tells us about the spin being spin-up or spin down. And $\hat{x}$, flips the state between up and down. However, because our bits are quantum states (vectors), they can also point in other directions besides up and down - this is a key difference between quantum and classical computers!

Consider the last operator/gate we will need, $\hat{U}^H$ (the Hadamard gate):

$$\begin{aligned} \hat{U}^H &= \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \\ &= \frac{1}{\sqrt{2}} (\hat{x} + \hat{z}) \end{aligned}$$

Applying this operator to a state does something interesting:

$$\begin{aligned} \hat{U}^H |0\rangle &= \frac{1}{\sqrt{2}} (\hat{x} + \hat{z}) |0\rangle \\ &= \frac{1}{\sqrt{2}} (|1\rangle + |0\rangle) \\ \hat{U}^H |1\rangle &= \frac{1}{\sqrt{2}} (\hat{x} + \hat{z}) |1\rangle \\ &= \frac{1}{\sqrt{2}} (|0\rangle - |1\rangle) \end{aligned}$$

We’ve now created **superposition** states! Play around with these objects in the following python cell.

```
[1]: import numpy as np

# define our qubit states
s0 = np.array([[1],[0]])
s1 = np.array([[0],[1]])

# define z
z = np.array([[1,0],[0,-1]])
print(" z operator: \n", z)

# define x
x = np.array([[0,1],[1,0]])
print(" x operator: \n", x)

# define Uh
Uh = np.array([[1,1],[1,-1]])/np.sqrt(2)
print(" Uh operator: \n", Uh)

print("\n z|0>: \n", z@s0)
print("\n z|1>: \n", z@s1)

print("\n x|0>: \n", x@s0)
print("\n x|1>: \n", x@s1)

print("\n Uh|0>: \n", Uh@s0)
print("\n Uh|1>: \n", Uh@s1)

print("\n |0> |1>: \n", np.kron(s0,s1))
print("\n z z      : \n", np.kron(z,z))
```

```
z operator:
[[ 1  0]
 [ 0 -1]]
x operator:
[[0 1]
 [1 0]]
Uh operator:
[[ 0.70710678  0.70710678]
 [ 0.70710678 -0.70710678]]
```

```
z|0>:
[[1]
 [0]]
```

```
z|1>:
[[ 0]
 [-1]]
```

```
x|0>:
[[0]
 [1]]
```

```
x|1>:
[[1]
 [0]]
```

```
Uh|0>:
[[0.70710678]
 [0.70710678]]
```

```
Uh|1>:
[[ 0.70710678]
 [-0.70710678]]
```

```
|0> |1>:
[[0]
 [1]
 [0]
 [0]]
```

```
z z      :
[[ 1  0  0  0]
 [ 0 -1  0  0]
 [ 0  0 -1  0]
 [ 0  0  0  1]]
```

4 Question:

In the next cell: explain what $\hat{U}^H$ does to the states, $|0\rangle$ and $|1\rangle$ in terms of $\hat{x}$:

$\hat{U}^H$ maps the computational-basis states ($\hat{z}$ eigenstates) to the eigenstates of $\hat{x}$:

- $\hat{U}^H |0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \equiv |+\rangle$, which is the 1 eigenstate of $\hat{x}$: $\hat{x}|+\rangle = +|+\rangle$.
- $\hat{U}^H |1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \equiv |-\rangle$, which is the -1 eigenstate of $\hat{x}$: $\hat{x}|-\rangle = -|-\rangle$.

So $\hat{U}^H$ rotates the $\hat{z}$ -basis into the $\hat{x}$ -basis. Equivalently, $\hat{U}^H \hat{x} \hat{U}^H = \hat{z}$, so applying $\hat{U}^H$ before a $\hat{z}$ -basis measurement is the same as measuring $\hat{x}$.

4.1 Adiabatic Evolution of Ising Hamiltonian with Quantum Circuit

We discussed that the adiabatic principle can be used to obtain the ground state of a difficult problem (e.g., Ising Hamiltonian) by starting with an easy problem (which for which we know the solution) and slowly moving toward the difficult solution. This means that we need to *drive our system forward in time* with a time-dependent Hamiltonian. This is generally a difficult task, but we can leverage a quantum computer to do this for us!

By choosing $\hat{H}_0 = -\sum_i \sigma_i^x$, recall that the ground state of the Ising model can be prepared by the following sequence of time evolutions:

$$|\psi(T)\rangle \approx U(4\Delta t, 3\Delta t)U(3\Delta t, 2\Delta t)U(2\Delta t, \Delta t)U(\Delta t, 0) |\psi(0)\rangle$$

which is,

$$|\psi(T)\rangle \approx e^{-i\left(\frac{1}{4}\hat{H}_0 + \frac{3}{4}\hat{H}_1\right)\Delta t} e^{-i\left(\frac{1}{2}\hat{H}_0 + \frac{1}{2}\hat{H}_1\right)\Delta t} e^{-i\left(\frac{3}{4}\hat{H}_0 + \frac{1}{4}\hat{H}_1\right)\Delta t} e^{-i\hat{H}_0\Delta t} |\psi(0)\rangle,$$

where $N\Delta t = T$.

And again, we will approximate this a bit more by breaking up each time step into each operator:

$$|\psi(T)\rangle \approx e^{-i\frac{1}{4}\hat{H}_0\Delta t} e^{-i\frac{3}{4}\hat{H}_1\Delta t} e^{-i\frac{1}{2}\hat{H}_0\Delta t} e^{-i\frac{3}{2}\hat{H}_1\Delta t} e^{-i\frac{3}{4}\hat{H}_0\Delta t} e^{-i\frac{1}{4}\hat{H}_1\Delta t} e^{-i\hat{H}_0\Delta t} |\psi(0)\rangle,$$

Here the basic idea is to use a quantum computer to do this time evolution for us. We will use the fact that Qiskit defines for us the following gates (or operations):

Single qubit gates (applied to a given qubit n): - $h(n)$: $\hat{U}^H$ - $rx(, n)$: $e^{-i\frac{\theta}{2}\hat{x}}$ - $rz(, n)$: $e^{-i\frac{\theta}{2}\hat{z}}$

Two qubit gates (applied to two given qubits, n and m): - $rz(, n, m)$: $e^{-i\frac{\theta}{2}\hat{z}_n \otimes \hat{z}_m}$

4.1.1 Trotterization

One trick we will use is **Trotterization**. For 2 operators $\hat{o}_1$ and $\hat{o}_2$, we can say the following two things: - if $\hat{o}_1\hat{o}_2 = \hat{o}_2\hat{o}_1$, then $e^{a(\hat{o}_1+\hat{o}_2)} = e^{a\hat{o}_1}e^{a\hat{o}_2}$ - if $\hat{o}_1\hat{o}_2 \neq \hat{o}_2\hat{o}_1$ but $a \ll 1$, then $e^{a(\hat{o}_1+\hat{o}_2)} \approx e^{a\hat{o}_1}e^{a\hat{o}_2}$

Where we say that $\hat{o}_1$ and $\hat{o}_2$ commute if $\hat{o}_1\hat{o}_2 = \hat{o}_2\hat{o}_1$

4.2 Initialization

From our discussion before, we saw that applying a Hadamard gate to the $|0\rangle$ state created the $|+\rangle$ state, which happened to be the $+1$ eigenstate of σ_x : - $\sigma_x |+\rangle = |+\rangle$

As such, we can initialize our system in the ground state of $\hat{H}_0$ by simply applying a Hadamard to each qubit.

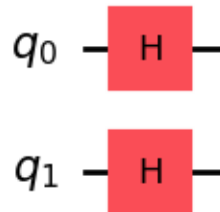
For this notebook, we'll use Qiskit, which will need to be installed into your environment:

```
conda install -c conda-forge qiskit qiskit-aer pylatexenc
```

```
[2]: import numpy as np
import matplotlib.pyplot as plt
from qiskit import *
from qiskit.visualization import plot_histogram
from qiskit import QuantumCircuit
```

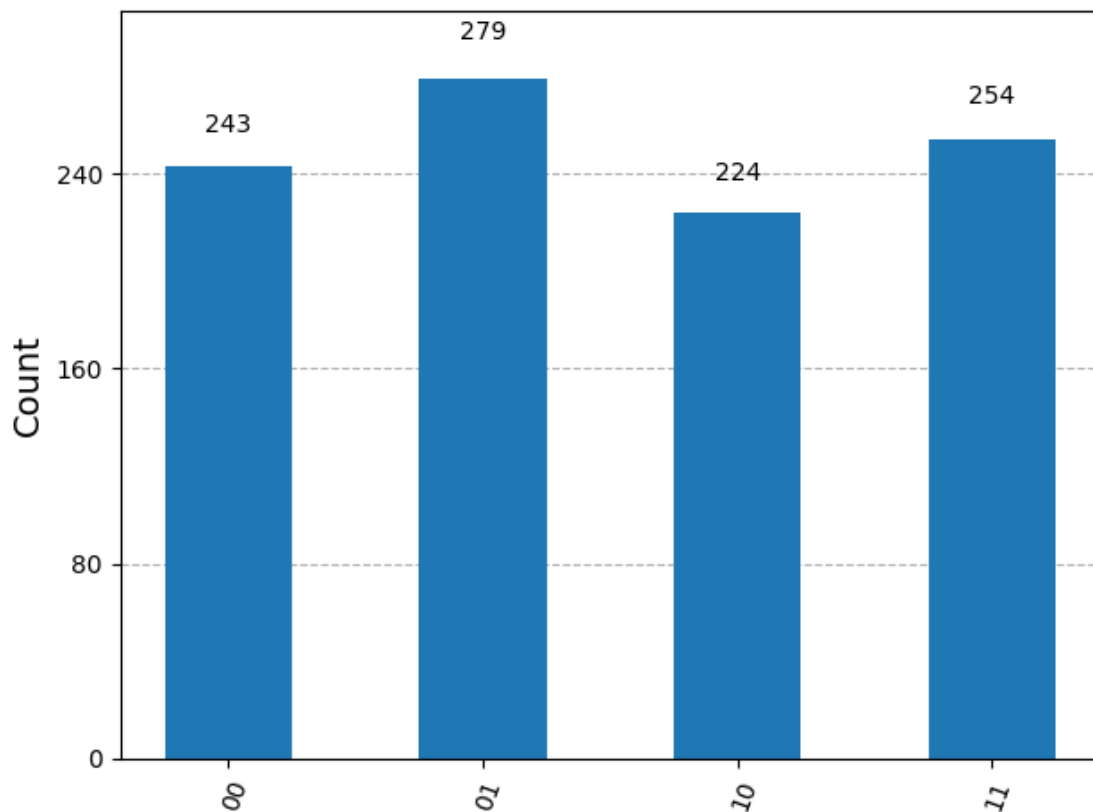
```
circ = QuantumCircuit(2,0)
circ.h(0)
circ.h(1)
circ.draw('mpl')
```

[2]:



```
[3]: from qiskit_aer.primitives import SamplerV2 as Sampler
sampler = Sampler()
circ.measure_all()
result = sampler.run([circ], shots=1000).result()
counts = result[0].data.meas.get_counts()
plot_histogram(counts)
```

[3]:



4.3 First time step

How do we implement $e^{-i\hat{H}_0\Delta t}$ with gates? Let's make things more concrete first, and consider a simple 2-site Ising problem. - $\hat{H}_0 = -\sigma_0^x - \sigma_1^x$ - $\hat{H}_1 = J\sigma_0^z\sigma_1^z + \mu\sigma_0^z + \mu\sigma_1^z$ - $J = 1$ - $\mu = .1$

or by simplifying the notation a bit: - $\hat{H}_0 = -\hat{x}_0 - \hat{x}_1$ - $\hat{H}_1 = J\hat{z}_0\hat{z}_1 + \mu\hat{z}_0 + \mu\hat{z}_1$ - $J = 1$ - $\mu = .1$

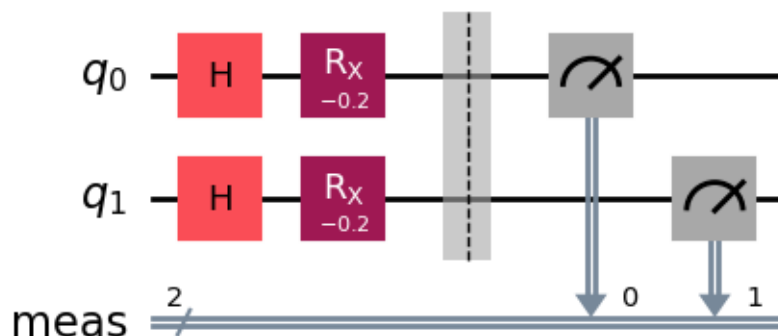
Because $\hat{x}_0$ and $\hat{x}_1$ act on different qubits, they commute, meaning that we can break this first time step up into 2 sequential steps exactly:

$$e^{i(\hat{x}_0+\hat{x}_1)\Delta t} = e^{i\hat{x}_0\Delta t}e^{i\hat{x}_1\Delta t}$$

Let's choose a small time step, say $\Delta t = .1$. Remembering the fact that $\text{rx}(,n): e^{-i\frac{\theta}{2}\hat{x}_n}$, we can use the following quantum circuit to implement this time step:

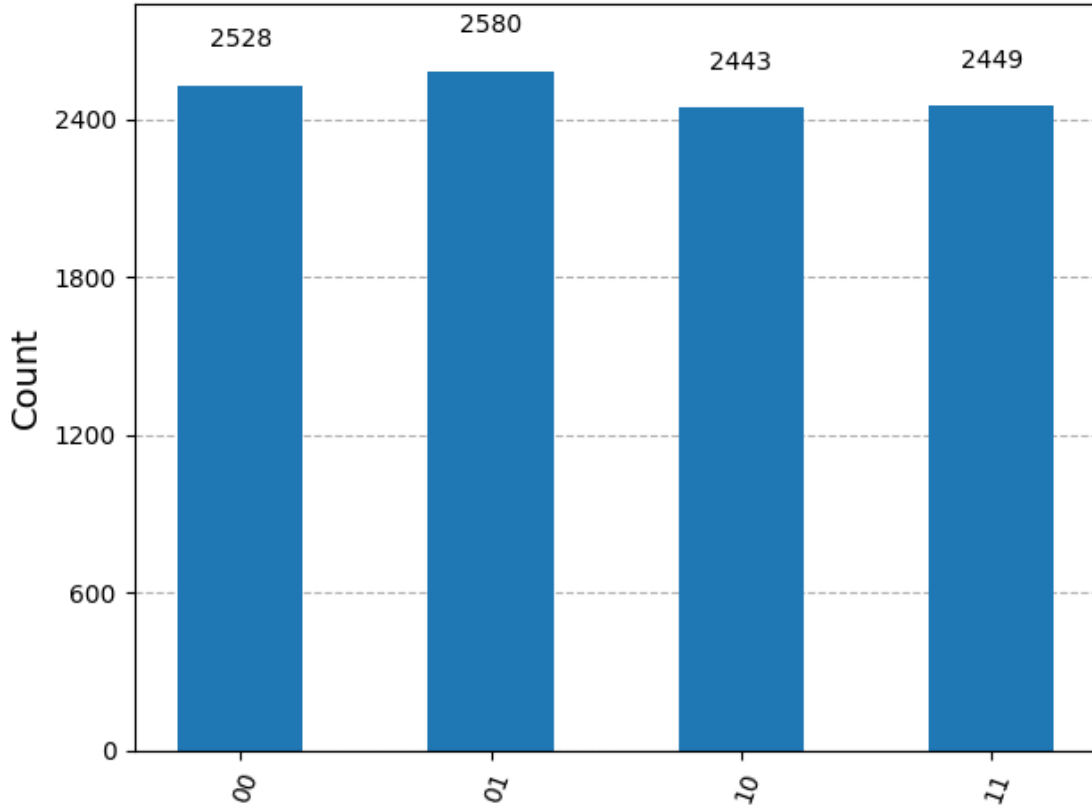
```
[4]: circ = QuantumCircuit(2,0)
      circ.h(0)
      circ.h(1)
      circ.rx(-2*.1,1)
      circ.rx(-2*.1,0)
      circ.measure_all()
      circ.draw(output = 'mpl', style="iqp")
```

[4]:



```
[5]: nshots = 10000
      plot_histogram(sampler.run([circ], shots=nshots).result()[0].data.meas.
                     ↪get_counts())
```

[5]:



5 Question:

Explain why the measured probability stayed the same in the above plot:

After the Hadamards each qubit is in $|+\rangle$, which is an eigenstate of $\hat{x}$ with eigenvalue $+1$. The gate $\text{rx}(\theta) = e^{-i(\theta/2)\hat{x}}$ is just a rotation about the $\hat{x}$ -axis, so

$$\text{rx}(\theta) |+\rangle = e^{-i\theta/2} |+\rangle,$$

ex) the state is unchanged up to a global phase. Since the measurement is in the $\hat{z}$ -basis, $|+\rangle$ still gives probability $\frac{1}{2}$ for each outcome, and the two-qubit histogram stays uniform over the four bitstrings. The $e^{-i\hat{H}_0\Delta t}$ step has no effect on the ground state of $\hat{H}_0$.

5.1 Second time step

Now we must implement the following operator: $e^{-i(.75\hat{H}_0+.25\hat{H}_1)\Delta t}$.

Unfortunately, $zx \neq xz$ and so we can't exactly trotterize this. However, because $\Delta t \ll 1$, we can approximate this in a product form, and this will become increasingly more accurate as we take smaller time steps.

$$\hat{H}_0 = -\hat{x}_0 - \hat{x}_1$$

$$\hat{H}_1 = J\hat{z}_0\hat{z}_1 + \mu\hat{z}_0 + \mu\hat{z}_1$$

$$e^{-i\left(-\frac{3}{4}\hat{x}_0 - \frac{3}{4}\hat{x}_1 + \frac{1}{4}J\hat{z}_0\hat{z}_1 + \frac{1}{4}\mu\hat{z}_0 + \frac{1}{4}\mu\hat{z}_1\right)\Delta t} \approx e^{i\frac{3}{4}\hat{x}_0\Delta t} e^{i\frac{3}{4}\hat{x}_1\Delta t} e^{-i\frac{1}{4}J\hat{z}_0\hat{z}_1\Delta t} e^{-i\frac{1}{4}\mu\hat{z}_0\Delta t} e^{-i\frac{1}{4}\mu\hat{z}_1\Delta t}$$

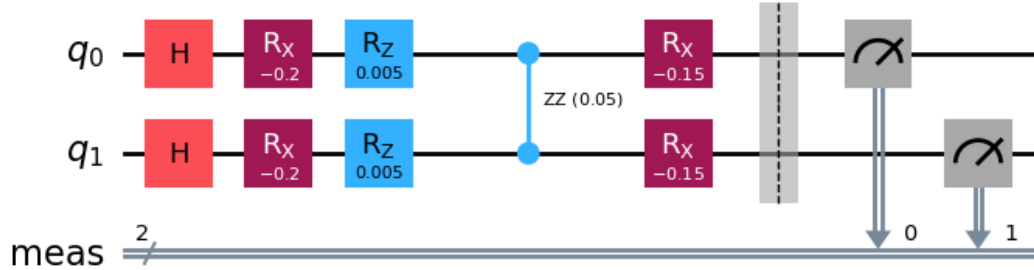
Notice here that we are using a new gate, RZZ. This is a 2-qubit gate that rotates about the product of two $\hat{z}$ operators.

```
[6]: circ = QuantumCircuit(2,0)
# initialize
circ.h(0)
circ.h(1)

# time step 1
circ.rx(-2*.1,1)
circ.rx(-2*.1,0)

# time step 2
circ.rz(2 * .1 * .25 * .1,1)
circ.rz(2 * .1 * .25 * .1,0)
circ.rzz(2 * 1. * .25 * .1,0,1)
circ.rx(-2 * .75 * .1,1)
circ.rx(-2 * .75 * .1,0)
circ.measure_all()
circ.draw(output = 'mpl')
```

[6]:



From here we can recognize the pattern and start to generalize with a function!

5.2 Automate the circuit building

Because each step will have the same gates, but different angles, we can simply write a function to create the circuit.

```
[7]: def form_circuit(N, T, ham):
    nq = ham.N
    dt = T / N
    circ = QuantumCircuit(nq, 0)

    # Initialize in ground state of H_0:  $|+\rangle^{\otimes n}$ 
    for i in range(nq):
        circ.h(i)

    for k in range(N):
        # weight on H_0
        a = 1.0 - k / N
        # weight on H_1
        b = k / N

        #  $e^{-i dt b H_1}$ 
        if b > 0.0:
            # local mu z_i:  $rz(2 * \mu_i * b * dt)$ 
            for i in range(nq):
                if ham.mu[i] != 0.0:
                    circ.rz(2.0 * ham.mu[i] * b * dt, i)
            #  $J_{ij} z_i z_j$ :  $rzz(2 * J_{ij} * b * dt)$ 
            for (i, j) in ham.G.edges():
                Jij = ham.G.edges[i, j]['weight']
                circ.rzz(2.0 * Jij * b * dt, i, j)

        #  $e^{-i dt a H_0} = e^{+i dt a \sum_i x_i}$ 
        #  $rx(\theta) = e^{-i (\theta/2) x}$ , so  $\theta = -2 * a * dt$ 
        if a > 0.0:
            for i in range(nq):
                circ.rx(-2.0 * a * dt, i)

    circ.measure_all()
    return circ
```

5.3 Demonstrations

Try out different numbers of steps and different Hamiltonians. - $\hat{H}(t) = (1 - f(t))\hat{H}_0 + f(t)\hat{H}_1$

Let's build a random 1D Ising Hamiltonian:

```
[8]: %load_ext autoreload
%autoreload 2

import networkx as nx
import matplotlib.pyplot as plt
import random
import montecarlo
```

```

N = 3
G = nx.Graph()
G.add_nodes_from([i for i in range(N)])
G.add_edges_from([(i,(i+1)% G.number_of_nodes() ) for i in range(N)])

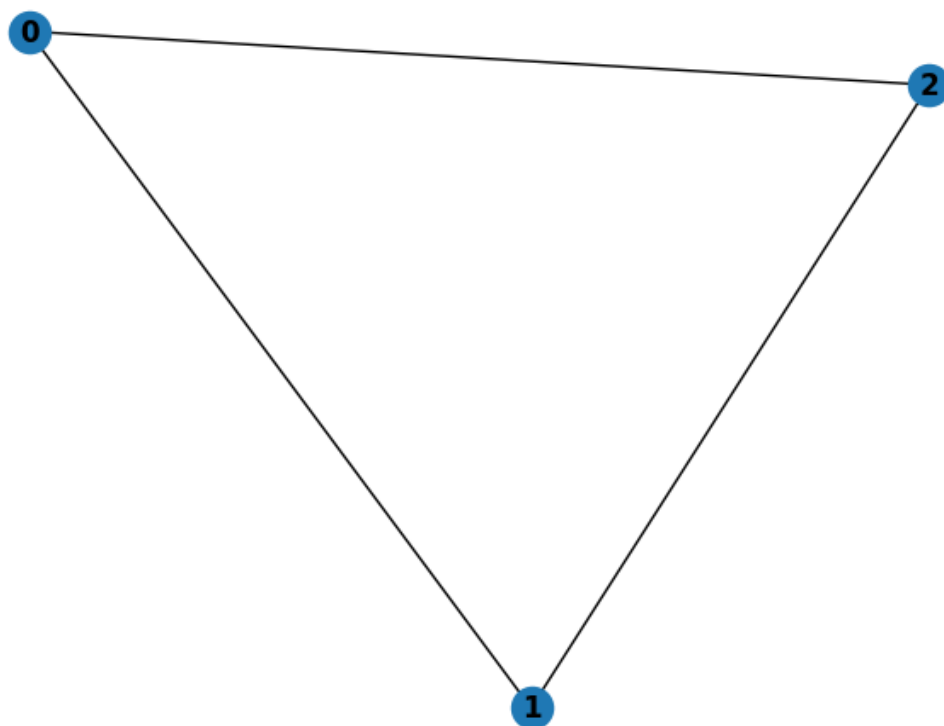
# let's add some random weights to the edges.
G.edges[(0, 1)]["weight"] = -0.5392900545734278
G.edges[(0, 2)]["weight"] = 0.3694282570558236
G.edges[(1, 2)]["weight"] = -0.37530144644718777

# # alternatively, you can do this using the random package.
# for e in G.edges:
#     G.edges[e]['weight'] = (random.random()*2 - 1)
#     print(e[0], e[1], G.edges[e]["weight"])

# Now Draw the graph.
plt.figure(1)
nx.draw(G, with_labels=True, font_weight='bold')
plt.show()

ham = montecarlo.IsingHamiltonian(G)
# let's add local mu values
for i in range(ham.N):
    ham.mu[i] = random.random()

```



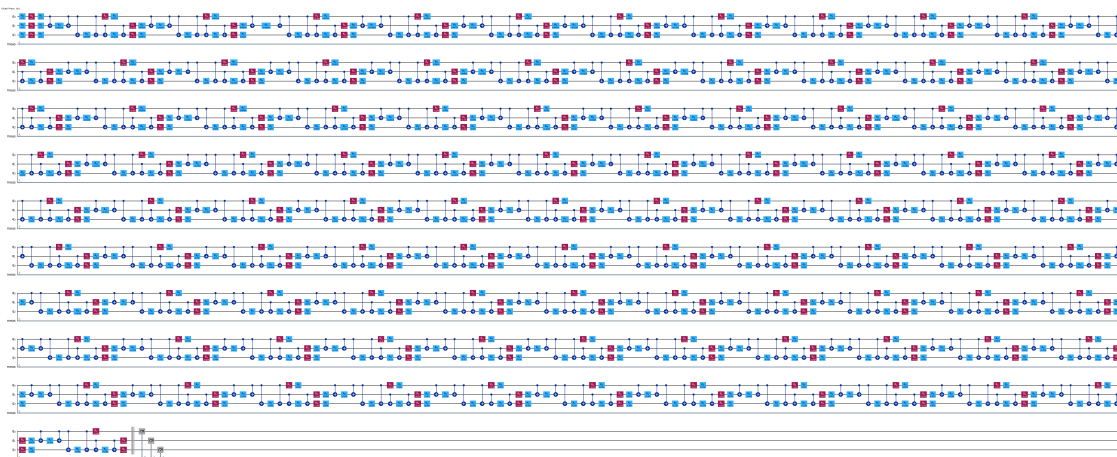
```
[9]: from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

T = 20
N = 100

# This is the function you need to implement
circ = form_circuit(N, T, ham)

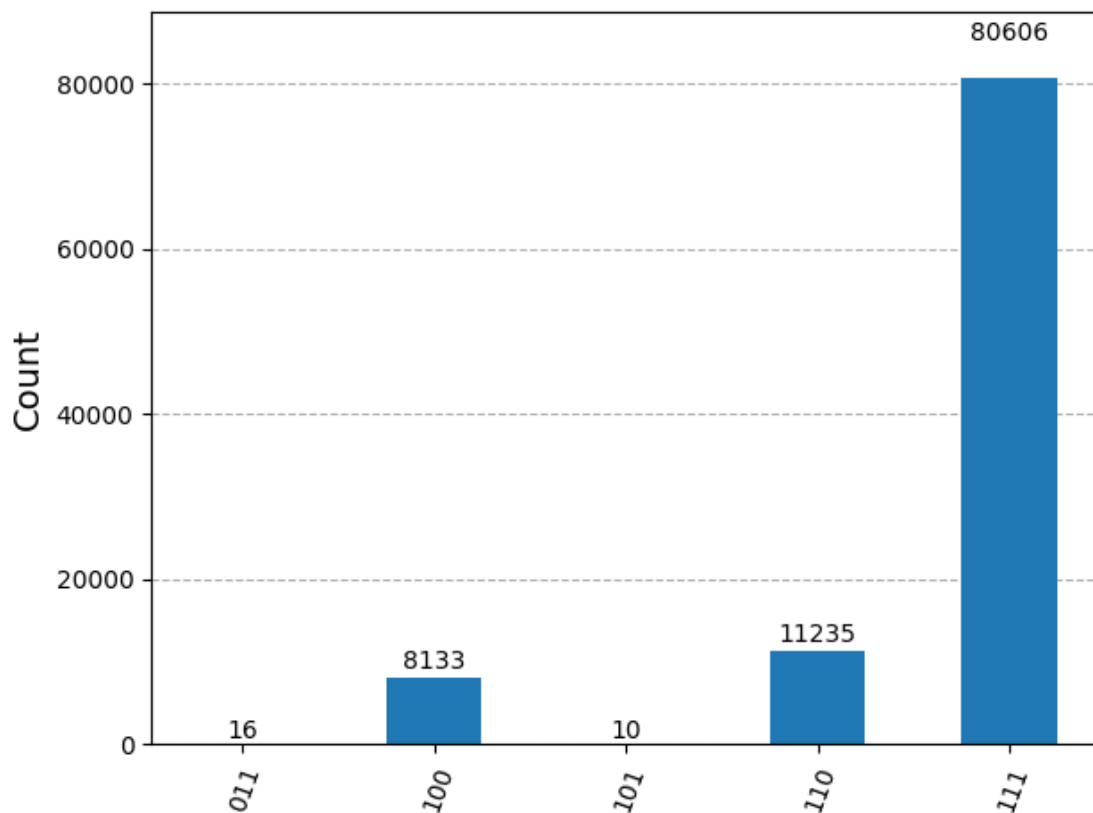
pm = generate_preset_pass_manager(optimization_level=3, basis_gates=["rz", "
    ↪ "rx", "cx"])
circ = pm.run(circ)
circ.draw(output="mpl", fold=120)
```

[9]:



```
[10]: nshots = 100000
      plot_histogram(sampler.run([circ], shots=nshots).result()[0].data.meas.
                    ↪ get_counts())
```

[10]:



```
[11]: samples = sampler.run([circ], shots=nshots).result()[0].data.meas.get_counts()
      max(samples, key=samples.get)
```

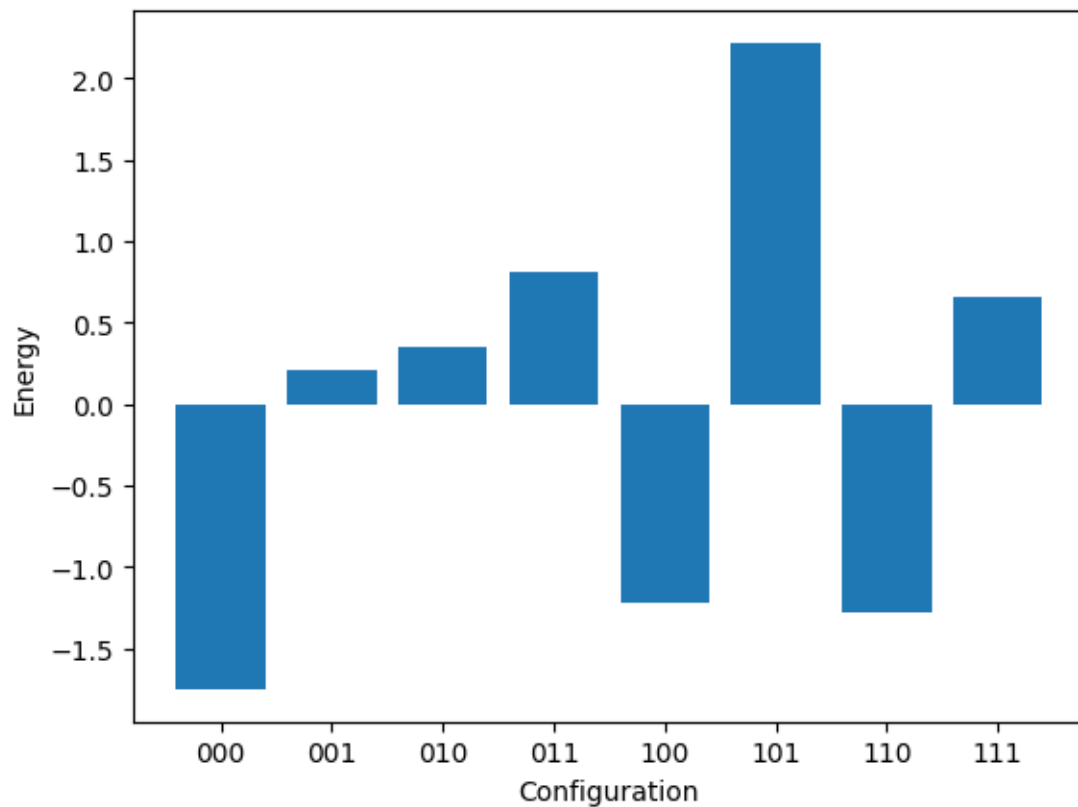
```
[11]: '111'
```

6 Validation

How well does this match the exact result?

```
[12]: import copy as cp
      configs = []
      energies = []
      conf = montecarlo.BitString(ham.N)
      for i in range(2**ham.N):
          conf.set_integer_config(i)
          configs.append(str(conf))
          energies.append(ham.energy(conf))
      plt.bar(configs, energies)
      plt.xlabel('Configuration')
      plt.ylabel('Energy')
```

```
[12]: Text(0, 0.5, 'Energy')
```



Note: Since 0 and 1 have opposite meanings between the classical and quantum examples (0 was spin down classically, while $|1\rangle$ was the negative eigenvalue of $\hat{z}$), the bitstrings should be opposite one another. For example, the classical state 0101 corresponds to the 1010 state. However, Qiskit prints the bitstrings from right to left, whereas we print ours from left to right, so that means 1010 corresponds to 1010, and 101 corresponds to 010.

7 Question:

What are the probabilities for measuring the correct bitstring using the following number of time steps (i.e., values of `n_steps`): 1. `n_steps = 3` 1. `n_steps = 4` 1. `n_steps = 8` 1. `n_steps = 10`

```
[13]: # Sweep over n_steps and look at the probability of measuring the correct
# ground-state bitstring. Hold T fixed and vary N.
T = 20
nshots = 100000

# Find the exact classical ground state (the bitstring with lowest energy).
conf = montecarlo.BitString(ham.N)
best_int, best_E = 0, float("inf")
for i in range(2 ** ham.N):
    conf.set_integer_config(i)
    E = ham.energy(conf)
    if E < best_E:
        best_E, best_int = E, i
conf.set_integer_config(best_int)
classical_gs = str(conf)

# Convert to the Qiskit measurement string.
# Classical convention: 0 = spin-down, 1 = spin-up (s_i = 2 b_i - 1).
# Quantum convention here: |1> is the -1 eigenstate of z, so spin-down -> |1>.
# So we flip every bit, then reverse because Qiskit prints qubits right-to-left.
quantum_gs = "".join("1" if c == "0" else "0" for c in classical_gs[::-1])
print(f"classical ground state: {classical_gs}    E = {best_E:.4f}")
print(f"corresponding Qiskit bitstring: {quantum_gs}\n")

pm = generate_preset_pass_manager(optimization_level=3, basis_gates=["rz", "rx", "cx"])
results = {}
for n_steps in [3, 4, 8, 10]:
    circ = form_circuit(n_steps, T, ham)
    circ_t = pm.run(circ)
    counts = sampler.run([circ_t], shots=nshots).result()[0].data.meas.
    get_counts()
    p = counts.get(quantum_gs, 0) / nshots
    results[n_steps] = p
```

```
print(f"n_steps = {n_steps:>3d}: P({quantum_gs}) = {p:.4f}")
```

```
classical ground state: 000    E = -1.7507  
corresponding Qiskit bitstring: 111
```

```
n_steps = 3: P(111) = 0.1659  
n_steps = 4: P(111) = 0.4525  
n_steps = 8: P(111) = 0.3019  
n_steps = 10: P(111) = 0.1126
```

7.1 Answer

The exact probabilities depend on the random μ draws used to build the Ising Hamiltonian, so the numbers shift between sessions. For the run executed above ($T = 20$, 100k shots, classical ground state 000, Qiskit string 111), I found:

n_steps	$\Delta t = T/N$	$P(\text{ground state})$
3	6.67	0.17
4	5.00	0.45
8	2.50	0.30
10	2.00	0.11